

OWLPAY

Inspect the work. Release the payment.

Agent-native verification and payment infrastructure for GitHub bounties

A project owner locks the reward. The Owl Agent verifies the pull request, purchases extra checks through x402 when needed, and releases the payment on GOAT mainnet only after the evidence passes.

Detailed Product Description, Technical Architecture and MVP Roadmap

Version 1.0 | August 2026

1. Executive Summary

OwlPay is an agent-native payment and verification system designed for GitHub development bounties. A project owner creates a task and locks the reward in a smart contract on GOAT mainnet. When a developer submits a pull request, the Owl Agent evaluates the work against predefined acceptance criteria, code changes, test results and repository evidence.

When deeper verification is necessary, the agent purchases a paid testing or security report through x402. If all mandatory requirements are satisfied, the contract releases the reward to the developer. If there are issues, the funds remain locked and the developer receives clear revision feedback. Uncertain cases are escalated to human review.

In one sentence: OwlPay turns GitHub bounty requirements into verifiable evidence and turns that evidence into an automatic on-chain payment decision.

2. Product Identity

Element	Definition
Product name	OwlPay
Core agent	Owl Agent
Paid verifier	OwlScan402
Smart contract	OwlPayBounty.sol
Primary network	GOAT Network
Primary use case	GitHub development bounty verification and settlement
Slogan	Inspect the work. Release the payment.

The owl is positioned as a professional detective rather than a playful mascot. It represents attention, evidence gathering, judgment and continuous monitoring. The visual identity should remain suitable for a financial developer tool: dark navy, warm gold, restrained illustrations and clear data presentation.

3. The Problem

3.1 Developer risk

- The reward may not be funded when the developer starts work.

- Payment can be delayed after delivery.
- New requirements may be introduced after the pull request is submitted.
- A completed task may be rejected without clear technical evidence.

3.2 Project owner risk

- The submitted code may not satisfy the actual task.
- Tests may fail or existing features may regress.
- The pull request may introduce a security issue or hidden secret.
- Manual review is slow and expensive for small bounty amounts.

3.3 OwlPay solution

The reward is funded before work begins and remains locked under transparent contract rules. The delivery is evaluated against evidence. Payment is released only after the mandatory requirements pass. This gives developers payment assurance and gives project owners lower-cost, structured verification.

4. Why the Technology Is Necessary

Technology	Why it is required
AI Agent	Natural-language requirements must be converted into a verification plan. The agent decides which evidence is sufficient, whether a paid check is worth its cost and whether the result should be approved, revised or escalated.
GOAT mainnet	The reward must be visibly funded, locked and settled under programmatic rules without relying on the project owner to send payment manually.
x402	The agent needs a native way to buy a testing, security or code-analysis result only when that result is necessary. Pay-per-call verification is more appropriate than opening accounts and subscriptions.
Smart contract	The agent cannot freely move arbitrary user funds. The contract limits settlement to the registered bounty, amount and developer address.
GitHub evidence	Pull requests, commits, diffs, checks and test results make the first use case objective enough for an MVP.

5. Target MVP Use Case

The MVP supports GitHub development bounties only. It does not attempt to judge design, translation, articles or other subjective freelance work.

Example bounty

Task: Add a GET /health endpoint

Requirement 1: Return HTTP 200

Requirement 2: Return {"status":"ok"}

Requirement 3: Existing tests must pass

Requirement 4: Add at least one new test

Requirement 5: Update README

Reward: 20 USDC

Verification budget: 0.50 USDC

6. User Roles

Role	Responsibilities
Project Owner	Creates the bounty, defines acceptance criteria, sets reward and verification budget, funds the contract and reviews the final evidence.
Developer	Selects the bounty, completes the work, submits the pull request and wallet address, receives feedback and receives payment after approval.
Owl Agent	Collects evidence, creates the verification plan, decides whether paid verification is needed, evaluates results and requests settlement or revision.
OwlScan402	Provides a paid, commit-bound verification report after receiving an x402 payment.
Human Reviewer	Handles subjective, conflicting or low-confidence cases. The MVP must never force automatic settlement when evidence is uncertain.

7. End-to-End Product Flow

1. Project owner creates a GitHub bounty and defines measurable acceptance criteria.
2. Project owner funds the bounty in the OwlPay smart contract on GOAT mainnet.
3. Developer completes the task and submits a pull request and payout wallet.
4. Backend collects repository, issue, pull request, commit, diff and CI evidence.

5. Owl Agent maps each acceptance criterion to a verification method.
6. When deeper analysis is justified, the agent calls OwlScan402.
7. OwlScan402 returns HTTP 402 with the required price.
8. The agent checks the verification budget, service allowlist and duplicate purchases.
9. The agent completes the x402 payment and receives the paid report.
10. The agent returns APPROVE, REVISION_REQUIRED or HUMAN_REVIEW with evidence.
11. When approved under policy, the contract releases the reward to the developer.
12. Dashboard displays funding, verification payment and settlement transactions.

8. Agent Decision Policy

Decision	Required conditions	Financial result
APPROVE	All mandatory criteria passed, no blocking issue, sufficient confidence and valid verification evidence.	Reward is released to the registered developer.
REVISION_REQUIRED	A mandatory criterion failed or a correctable issue remains.	Funds remain locked and detailed feedback is returned.
HUMAN_REVIEW	Low confidence, subjective criteria, conflicting evidence or reward above automation threshold.	No automatic payout occurs.

Suggested automatic settlement gate: decision is APPROVE, confidence is at least 0.90, every mandatory criterion is PASSED and blockingIssues is empty.

9. Main Components

9.1 OwlPay Web App

- Wallet connection
- Bounty creation
- Open bounty list
- Pull request submission
- Owner and developer dashboards
- Verification report view
- Transaction progress and explorer links

9.2 OwlPayBounty smart contract

- Create and fund bounty
- Register delivery
- Store task, submission and verification hashes
- Authorize settlement agent
- Release payment once

- Refund expired unresolved bounty
- Pause emergency operations

9.3 Owl Agent

- Parse acceptance criteria
- Collect GitHub evidence
- Create verification plan
- Control x402 spending
- Interpret verification report
- Return structured decision
- Request policy-controlled settlement

9.4 OwlScan402

- Return 402 before payment
- Run controlled test and static analysis
- Bind result to repository and commit SHA
- Return structured signed report
- Reject invalid or replayed payment proof

10. Technical Architecture

Layer	Recommended stack
Frontend	Next.js, TypeScript, Tailwind CSS, wagmi, viem, TanStack Query
API	Node.js, TypeScript, Fastify, Zod, PostgreSQL, Prisma
Smart contracts	Solidity, Foundry, OpenZeppelin
Agent	TypeScript, structured JSON outputs, tool-based workflow, GOAT AgentKit
Verifier	Node.js, Docker sandbox, tests, linting and static analysis
Workspace	pnpm workspaces and Turborepo

10.1 Suggested monorepo

```
owlpay/
  apps/web
  apps/api
  apps/owlscan
  packages/contracts
```

packages/agent
 packages/shared
 packages/database
 docs/
 AGENTS.md
 pnpm-workspace.yaml
 turbo.json

11. Smart Contract Model

Stored field	Purpose
owner	Project owner who funded the bounty
developer	Registered recipient after submission
paymentToken	Allowed settlement token
rewardAmount	Amount locked for the developer
verificationBudget	Maximum amount the agent may spend on verification
deadline	Expiry time for unresolved bounty
taskHash	Hash of task and acceptance criteria
submissionHash	Hash of PR and commit evidence
verificationHash	Hash of the final report
status	Current lifecycle state

11.1 Required functions

- createBounty
- submitWork
- requestRevision
- approveSubmission
- releasePayment
- refundExpiredBounty
- cancelUnassignedBounty
- getBounty

11.2 Security controls

- Reentrancy protection

- Safe token transfers
- Emergency pause
- Single payout protection
- Deadline checks
- Authorized settlement role
- Registered developer enforcement
- Submission hash replay prevention
- No private key exposure in frontend
- Agent cannot send arbitrary transfers

12. Data Model

Entity	Core fields
User	walletAddress, githubUsername, createdAt
Repository	owner, name, githubUrl, createdBy
Bounty	onchainId, owner, developer, criteria, reward, budget, deadline, status
Submission	bountyId, pullRequestUrl, commitSha, developer, status
Verification	provider, price, paymentTx, report, reportHash, recommendation
AgentDecision	decision, confidence, summary, evidence, blockingIssues

Private model chain-of-thought must not be stored. The product stores only user-facing summaries, structured evidence and transaction records.

13. Business Model

Revenue source	Description
Settlement fee	A small percentage or minimum fixed fee from each successful bounty payment.
Verification marketplace margin	A small share of paid verification services purchased by the agent.
Team plans	Organization features, multiple repositories, reporting and policy controls.
Platform API	Integration for bounty platforms, DAOs and developer communities.

Growth loop: more bounties lead to more verification purchases, more settlements and more transaction-linked revenue.

14. MVP Scope Boundaries

Included

- One GOAT environment at a time
- One payment token
- GitHub development bounties
- One paid verifier
- Three decisions: approve, revision, human review
- Real bounty funding
- Real x402 verification payment
- Real developer settlement

Not included

- General freelance marketplace
- Design or article judgment
- Multiple chains
- Multiple payment tokens
- Complex dispute court
- Automatic merge
- Arbitrary user scripts
- Full reputation marketplace
- Mobile application

15. 14-Day MVP Roadmap

Day	Milestone
1	Validate GOAT RPC, wallet, AgentKit, x402 and contract deployment
2	Implement smart contract state model
3	Write contract tests and deploy to development network
4	Build wallet connection and bounty creation UI
5	Build bounty listing, detail and submission flow

6	Integrate GitHub API and pull request evidence
7	Implement criteria parser
8	Implement verification planner and decision schema
9	Build OwlScan service and controlled verification report
10	Add x402 merchant and payer flow
11	Connect Owl Agent to OwlScan402
12	Connect approval to smart contract settlement
13	Run low-value mainnet demo and harden limits
14	Polish UI, README, demo video and application form

16. Development Phases and Exit Criteria

Phase 0 - Technical validation

- RPC connected
- Wallet balance read
- Contract deployed
- AgentKit action executed
- x402 flow understood

Phase 1 - Smart contract

- Owner funds bounty
- Developer submits
- Unauthorized approval fails
- Authorized approval succeeds
- Duplicate payout fails
- Expired refund works

Phase 2 - Frontend

- Wallet connects
- Bounty created from UI
- Transaction confirmed
- Bounty visible
- PR submitted

Phase 3 - GitHub

- Valid PR fetched
- Wrong repository rejected
- Commit stored
- Changed files visible
- CI status visible

Phase 4 - Agent

- Structured JSON always valid
- Evidence attached to each criterion
- Missing criterion requests revision
- Low confidence escalates

Phase 5 - OwlScan402

- Unpaid request returns 402
- Paid request returns report
- Report bound to commit
- Invalid proof rejected

Phase 6 - End-to-end

- Funding, verification payment and payout visible
- Failed verification keeps funds locked
- Passing verification releases funds

17. Codex Build Instructions

Codex should be used in small, testable work packages. Do not ask it to build the entire system in one prompt. Every task must include acceptance criteria and must finish with formatting, type checking, tests and build verification.

17.1 AGENTS.md product block

OwlPay Development Instructions

OwlPay is an agent-native payment and verification system for GitHub bounties.

A project owner locks a reward in a smart contract on GOAT Network.

A developer submits a GitHub pull request.

The Owl Agent evaluates predefined criteria, code changes and test evidence.

When extra verification is required, the agent purchases a paid report through x402.

If all mandatory requirements pass, OwlPay releases the reward to the developer.

Safety rules:

- Never expose private keys to the frontend.

- The agent cannot send arbitrary transfers.
- Settlement must target the registered bounty and developer.
- Paid verification must remain within the bounty budget.
- Low-confidence decisions must go to human review.
- Run lint, typecheck, tests and build before completion.

17.2 First Codex prompt

Read AGENTS.md and create the initial OwlPay monorepo using pnpm workspaces and Turborepo.

Create apps/web, apps/api, apps/owlscan, packages/contracts, packages/agent, packages/shared and packages/database.

Use Next.js, Fastify, Zod, Foundry, Prisma and strict TypeScript.

Add root scripts for dev, build, lint, typecheck and test.

Add .env.example files without real secrets.

Do not implement business logic yet.

Run installation, formatting, linting, type checking, tests and builds. Fix all errors before finishing.

Report files created, commands executed, results and remaining manual setup.

18. Demo Scenario

13. Project owner creates a 20 USDC bounty for a health endpoint.
14. Owner funds the OwlPay contract on GOAT mainnet.
15. Developer submits an intentionally flawed pull request.
16. Owl Agent detects that a deeper security check is justified.
17. OwlScan402 responds with a 0.05 USDC payment requirement.
18. Owl Agent completes the x402 payment and receives the report.
19. The report finds the blocking issue and Owl Agent requests revision.
20. Developer submits a corrected commit.
21. Owl Agent verifies all mandatory evidence and approves.
22. OwlPay contract releases the reward to the developer.
23. Dashboard shows bounty funding, x402 payment and developer payout transactions.

The memorable jury moment: The Owl Agent hired a paid verifier through x402 and released the developer reward only after the evidence passed.

19. Success Metrics

Metric	MVP target
End-to-end completion	One complete bounty from funding to payout
Real transactions	At least bounty funding, x402 verification payment and

	developer payout
Decision transparency	Every mandatory criterion has evidence
Safety	No duplicate payout and no arbitrary agent transfer
Time to resolution	Automated result produced within a few minutes
Revenue demonstration	Platform fee visible in settlement breakdown

20. Final Positioning

OwlPay should not be presented as a general AI judge for every type of freelance work. It should be positioned as evidence-based settlement infrastructure for GitHub bounties. This narrow starting point makes the product technically credible, financially meaningful and easy to demonstrate.

Official pitch: OwlPay is an agent-native payment system that inspects GitHub bounty submissions, purchases additional verification through x402 when needed, and releases approved rewards on GOAT mainnet.